

Learned Tactic Policies versus a Hand-Written Heuristic for Size-Extrapolating Proof Search in Propositional Sequent Calculus: A Small CPU Study

Pyrite-61S
STEM seed corpus

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/tactic-prediction-on-a-synthetic-arithmetic-propositional-sequent-calculus>

Abstract—Neural tactic policies are routinely used to guide best-first proof search, but their benefit over a careful hand-written ordering is rarely measured in a setting where the baseline can be tuned and optimal proofs can be computed. We build a tiny propositional sequent calculus (invertible G3cp-style rules, tactic = which compound formula to decompose), generate 20000 provable training sequents with 3–10 connectives, and train a small transformer and a feature-based MLP to imitate optimal tactics (minimum proof size by dynamic programming). We then compare best-first search guided by the policies against breadth-first search (BFS) and a tuned hand heuristic (defer branching rules) within a budget of 100 expansions on sequents with 11–20, 21–40 and 41–70 connectives, with 5 seeds, all systems sharing one search engine. The result is largely negative: the transformer policy beats BFS in every test bin (e.g. solved rate 0.41 vs. 0.20 at 41–70 connectives) but loses clearly to the hand heuristic (0.73), and the gap does not shrink with size. Search with the policy’s cumulative negative log-probability as cost is the weak point: following the same transformer greedily, or ranking tactics by the policy instead of using $-\log p$, raises the 41–70 solved rate to 0.73, on par with the heuristic. A training-set sweep shows the opposite of the usual trend: the transformer trained on 1000 sequents extrapolates better (0.52 at 41–70) than the one trained on 20000 (0.41), but still below the heuristic. In a post-hoc (not pre-registered) variant, the feature MLP with rank cost reaches 0.88 and exceeds the heuristic. All conclusions are limited to this synthetic, small-scale setting.

Index Terms—automated theorem proving, tactic prediction, best-first search, sequent calculus, length generalisation, negative result

I. Introduction

Learned tactic policies are a core component of modern neural theorem provers: a language model proposes the next proof step and a best-first or tree search decides which proof states to expand [1]–[4]. Earlier work used neural networks to rank premises or clauses for classical provers [5], [6] or to guide connection-tableau search with reinforcement learning [7], and incremental learning with hindsight relabelling has been used to learn provers from

few proofs [8]. In these systems, the learned component is evaluated on large libraries where a hand-written alternative is hard to build and optimal proofs are unknown, so it is difficult to say how much of a prover’s success is due to the learned ranking, to the search algorithm and cost function, or to the problem distribution.

Propositional sequent calculus offers a controlled microcosm. With invertible (G3-style) rules, every tactic preserves provability, so a prover never gets stuck; what a good tactic choice buys is a shorter proof and less search. Optimal proof size is computable for small sequents, a sound hand-written heuristic is known in principle (apply non-branching rules first so that context is not duplicated across branches), and sequents of arbitrary size are easy to generate. This makes it possible to ask a sharper question than “does the policy help?”:

On sequents larger than anything in training, and under a fixed budget of 100 expansions, does best-first search guided by a learned tactic policy find more proofs than breadth-first search and than a fairly tuned hand heuristic?

We pre-registered four hypotheses (Sec. IV): the policy beats BFS (H1) and the heuristic at the largest size (H2), the policy’s relative standing worsens with size (H3), and search beats greedy decoding of the policy (H4). Our results are mixed to negative. H1 is supported; H2, H3 and H4 are refuted. An exploratory diagnostic and ablations indicate that how the policy’s output is turned into a search priority matters more than the policy architecture, and that a feature-based MLP with a rank-based cost can exceed the heuristic in our setting, a post-hoc finding that we flag as such.

a) Contributions.: (i) A small, fully reproducible sequent-calculus prover, four random-provable-sequent generators and an optimal-proof labeller; (ii) a controlled comparison of BFS, a tuned hand heuristic, a feature MLP policy and a transformer policy under one search engine, with 5 seeds and three test size bins; (iii) ablations over the cost function, decoding mode, positional encoding, training-set size and budget, plus a diagnostic of how often the policies defer branching rules. We make no claim

Pyrite-61S is the codename of an autonomous study, not a person. Produced by the STEM autonomous AI research pipeline; the topic was chosen to cover the field. Every reported result is traced to logged runs in the linked repository and every reference was checked against arXiv, Crossref, DataCite or OpenAlex.

beyond this synthetic setting.

II. Related Work

a) Neural-guided theorem proving.: DeepMath [5] showed that neural networks can select premises for a first-order prover, and Loos et al. [6] used deep networks to guide clause selection in a saturation prover. Kaliszyk et al. [7] combined Monte-Carlo guidance with learned policy and value estimates for connection-tableau proving. GPT-f [1] introduced autoregressive transformer tactic generation with best-first search for Metamath; HyperTree Proof Search [2] adds an MCTS-style search over hypergraph proof states with online training, and BFS-Prover [3] revisits plain best-first tree search with large language models in Lean, arguing that a simple search with a strong policy can be competitive. LeanDojo [4] provides an open benchmark and retrieval-augmented tactic generation, and Aygün et al. [8] learn provers by incremental training with hindsight experience replay. All of these operate at a scale where the contribution of the learned policy is entangled with pretraining, retrieval and search design; the present study keeps the architecture of the pipeline (policy $\rightarrow$ cost $\rightarrow$ best-first search) but shrinks everything to a regime in which every component can be ablated.

b) Reasoning benchmarks and generalisation.: Synthetic tasks have long been used to probe whether neural sequence models reason: the mathematics dataset of Saxton et al. [9] reports weak extrapolation, Clark et al. [10] train transformers to answer rule-based deduction questions, and Anil et al. [11] study length generalisation of language models on reasoning tasks. Our test bins are larger than training formulas in the same sense, but the policy’s output is used inside a search rather than as a final answer. The transformer follows the standard architecture [12]; we use fixed sinusoidal positions and test learned positions as an ablation.

c) Gap.: To our knowledge, no prior work we examined reports, in one controlled calculus with computable optimal proofs, a tuned hand-written heuristic as a baseline, a learned policy, and a decomposition of the policy’s failures into ranking quality versus search-cost design; we did not conduct an exhaustive survey and state this only as the motivation for the study.

III. Method

A. Prover and tactics

Formulas are built from 5 propositional variables with $\neg, \wedge, \vee, \rightarrow$. A goal is a sequent $\Gamma \vdash \Delta$ of sets of formulas. We use the invertible, contraction-free rules of G3cp (the principal formula is removed and not copied):

$$\begin{aligned} \neg L : \Gamma, \neg A \vdash \Delta &\Rightarrow \Gamma \vdash A, \Delta \\ \wedge L : \Gamma, A \wedge B \vdash \Delta &\Rightarrow \Gamma, A, B \vdash \Delta \\ \rightarrow R : \Gamma \vdash A \rightarrow B, \Delta &\Rightarrow \Gamma, A \vdash B, \Delta \end{aligned}$$

$$\begin{aligned} \wedge R : \Gamma \vdash A \wedge B, \Delta &\Rightarrow \Gamma \vdash A, \Delta \text{ and } \Gamma \vdash B, \Delta \\ \vee L : \Gamma, A \vee B \vdash \Delta &\Rightarrow \Gamma, A \vdash \Delta \text{ and } \Gamma, B \vdash \Delta \\ \rightarrow L : \Gamma, A \rightarrow B \vdash \Delta &\Rightarrow \Gamma \vdash A, \Delta \text{ and } \Gamma, B \vdash \Delta \end{aligned}$$

A goal is closed by the axiom rule when a variable occurs on both sides; closure is applied automatically and costs no expansion. A tactic $a = (\text{side}, \varphi)$ names the compound formula φ to decompose (the rule follows from the side and the main connective); $\mathcal{A}(g)$ is the set of tactics of goal g . Because all rules are invertible, a goal is provable iff it is valid, every tactic leaves provability intact, and the choice of tactic affects only proof size. $\wedge R$, $\vee L$ and $\rightarrow L$ are branching: the remaining context is duplicated and decomposed in both branches.

B. Search engine (shared by all systems)

A search state is a list of open goals. One expansion pops a state, selects its first open goal g , and generates one child per tactic $a \in \mathcal{A}(g)$ (the child replaces g by the open subgoals of a). A proof is found when a generated child has no open goal; the search fails after 100 pops. All systems differ only in the priority of a child with parent priority P and step cost $c(g, a)$:

$$P(\text{child}) = P + c(g, a), \quad \text{ties broken towards deeper states, then in} \quad (1)$$

BFS (reimplemented): priority = depth (FIFO), the cost is ignored. Hand heuristic (reimplemented, ours): $c(g, a) = \text{rank}_g(a)$ in the ordering that places non-branching tactics before branching ones, then larger formulas first (variant large). Two alternatives (small: smaller first; right: right-side tactics first) were compared on a separate tuning bin (Sec. IV). Policy search (method): $c(g, a) = -\log \pi(a \mid g)$, so P is the negative log-likelihood of the tactic sequence, as in GPT-f-style best-first search [1]. The rank cost ablation uses $c(g, a) = \text{rank}_{\pi, g}(a)$ (rank of a under π , 0 for the top tactic), i.e. the same cost scale as the heuristic; the greedy ablation expands only the top-1 tactic and never backtracks.

C. Training labels

Let $s(g)$ be the minimum number of tactic applications of a proof of g : $s(g) = 0$ if g is an axiom and $s(g) = \min_{a \in \mathcal{A}(g)} (1 + \sum_{g' \in a(g)} s(g'))$ otherwise, computed by memoised recursion. The optimal set is $\mathcal{A}^*(g) = \arg \min_a (1 + \sum_{g'} s(g'))$. Training states are collected by walking a proof tree from each training sequent, following a uniformly chosen optimal tactic with probability 0.5000 and a uniformly random tactic otherwise (so that sub-optimal states are represented), keeping at most 30 states per sequent. The loss is the cross-entropy to the uniform distribution on $\mathcal{A}^*(g)$:

$$\begin{aligned} \neg L : \Gamma, \neg A \vdash \Delta &\Rightarrow \Gamma \vdash A, \Delta & \neg R : \Gamma \vdash \neg A, \Delta &\Rightarrow \Gamma, A \vdash \Delta \\ \wedge L : \Gamma, A \wedge B \vdash \Delta &\Rightarrow \Gamma, A, B \vdash \Delta & \vee R : \Gamma \vdash A \vee B, \Delta &\Rightarrow \Gamma \vdash A, \Delta \\ \rightarrow R : \Gamma \vdash A \rightarrow B, \Delta &\Rightarrow \Gamma, A \vdash B, \Delta & \rightarrow L : \Gamma, A \rightarrow B \vdash \Delta &\Rightarrow \Gamma \vdash A, \Delta \end{aligned}$$

$$\mathcal{L} = \frac{1}{|\mathcal{A}^*(g)|} \sum_{a \in \mathcal{A}^*(g)} \log \pi_\theta(a \mid g), \quad \pi_\theta(a \mid g) = \frac{\exp f_\theta(g)_a}{\sum_{a' \in \mathcal{A}(g)} \exp f_\theta(g)_{a'}} \quad (2)$$

D. Policies

Transformer policy (method). The goal is written in prefix notation as a token sequence (cls, left formulas, separators, right formulas) with token, side-segment and sinusoidal position embeddings; a pre-norm encoder with 3 layers, width 64, 4 heads and feedforward width 256 produces hidden states h_i . The logit of a tactic is $f_\theta(g)_a = w^\top \text{GELU}(Wh_{r(a)})$ where $r(a)$ is the position of the root token of the formula a names (a pointer-style head, so the output size follows the goal). Feature-MLP policy (baseline, reimplemented). Per tactic, a vector x_a holds the side $\times$ connective one-hot, formula size and depth, the overlap of its variables with the top-level variables of the same and the opposite side, and sequent-level context (number of compound formulas per side $\times$ connective, formula counts, total size); $f_\theta(g)_a = \text{MLP}(x_a)$ with two hidden layers of width 128. It uses no token sequence and hence has no positional-extrapolation problem, but sees only these hand-chosen features.

E. Sequent generators

We use four generators with equal probability, each followed by rejection to the requested size range (size = total number of connectives): (0) random sequents of 1–3 formulas with random sides, kept if valid by truth table; (1) a small valid “core” sequent (at most 4 connectives) with each variable substituted by a random formula; (2) as (1) plus one or two random distractor formulas added by weakening; (3) a random formula F and a formula F' obtained from F by 1 to 4 random equivalence rewrites (commutation, associativity, distributivity, De Morgan, double negation, contraposition, implication unfolding), giving the valid sequent $F \vdash F'$. Sequents that are already axioms are rejected and training sequents are distinct.

IV. Experimental Setup

a) Data and splits.: For each seed $s \in \{0, \dots, 4\}$ we generate 20000 distinct provable training sequents with 3–10 connectives and 1000 further held-out sequents of the same range for validation (top-1 accuracy of the policy against the optimal set, and the val task). The test tasks contain 150 problems each, drawn from RNGs seeded by (task, seed) and thus identical across systems: test_11_20, test_21_40 and test_41_70 (connective ranges 11–20, 21–40, 41–70, all strictly larger than any training sequent), plus val_3_10 (training range, held out) and tune_21_35, used only to choose the heuristic variant. Changing the seed changes the training set, the test problems and the model initialisation.

b) Metrics.: solved: fraction of problems proved within 100 expansions (primary, higher is better); mean_exp: mean number of expansions with failures counted as the budget (lower is better); val_top1 and val_loss: policy accuracy and loss on held-out training-range states. All systems use the same engine, budget, problems and metric definition.

TABLE I
Hand-heuristic variants on the tuning bin (21–35 connectives), five seeds, mean $\pm$ std.

Method	solved $\uparrow$	mean_exp $\downarrow$	n
Heuristic small	0.88 ± 0.03	32.03 ± 5.17	5
Heuristic large	0.99 ± 0.01	24.85 ± 2.01	5
Heuristic right	0.91 ± 0.01	28.06 ± 3.03	5

c) Training and tuning.: Both policies: AdamW (peak learning rate 0.002000, weight decay 0.01, one-cycle schedule), batch 128, 2000 steps, gradient clipping at 1; no hyperparameter of either policy was tuned, and none was ever selected on a test bin. The heuristic was tuned: three ordering variants were run on tune_21_35 with all five seeds (Table I), and the best (large) was used everywhere; this tuning bin is in the size regime of the test bins while the policies saw only training-range data, so the baseline has the advantage of access to the target size regime. Hardware: shared CPU, 2 threads per process, at most two processes at once. A policy is trained once per (system, seed) and its checkpoint is reused by all evaluation runs.

d) Pre-registered hypotheses and tests.: (H1) transformer-guided search solves more problems than BFS in each test bin; (H2) it solves more than the heuristic at 41–70; (H3) the transformer-minus-heuristic gap decreases monotonically from the 11–20 bin to the 41–70 bin; (H4) at 41–70 best-first search with the policy beats greedy decoding of the same policy. The tests are paired t -tests over the five seeds (the problems of a seed are identical for all systems), produced by rh compare and by method/analyze.py. With five seeds these tests have low power for small effects and the p -values are not corrected for multiple comparisons.

V. Results

Table II gives the main results (mean $\pm$ std over 5 seeds, 150 problems per task and seed) and Fig. 1 the per-bin solved rates, including two ablated variants discussed in Sec. VI. Paired statistics come from rh compare (paired t -test over seeds).

a) In the training size range: all guided systems are essentially perfect (solved: transformer 0.997, MLP 0.999, heuristic 1.000) while BFS already misses part of the problems (0.91); on this bin the heuristic’s advantage over the transformer is not significant (paired $p = 0.1778$). The held-out top-1 accuracy against the optimal tactic set is 0.892 for the transformer and 0.952 for the MLP, i.e. the transformer imitates optimal tactics less accurately than the MLP given the same number of steps.

b) H1 (policy > BFS): supported.: In all three test bins the transformer solves more problems than BFS: at 11–20 (0.82 vs. 0.47, $p = 2.232 \times 10^{-6}$), at 21–40 (0.52 vs. 0.27, $p = 2.033 \times 10^{-6}$) and at 41–70 (0.41 vs. 0.20, $p = 3.547 \times 10^{-6}$). BFS is a weak reference: it ignores

TABLE II

Main results: solved rate within 100 expansions and mean expansions (failures count as the budget). val is the training size range, the test tasks are larger (connective ranges in the task name). Rows for the tuning bin tune_21_35 omit the heuristic (its rows are in Table I) and are shown only for completeness; bold/underline in this table ignore the missing entry and are not meaningful there.

Method	Task	solved $\uparrow$	mean_exp $\downarrow$	n
BFS	val_3_10	0.91 $\pm$ 0.03	21.03 $\pm$ 3.42	5
Hand heuristic	val_3_10	1.00 $\pm$ 0.00	4.42 $\pm$ 0.28	5
Feature-MLP policy	val_3_10	1.00 $\pm$ 0.00	5.26 $\pm$ 0.76	5
Transformer policy	val_3_10	1.00 $\pm$ 0.00	5.99 $\pm$ 0.83	5
BFS	test_11_20	0.47 $\pm$ 0.03	63.33 $\pm$ 2.73	5
Hand heuristic	test_11_20	1.00 $\pm$ 0.00	11.31 $\pm$ 0.45	5
Feature-MLP policy	test_11_20	0.89 $\pm$ 0.02	27.16 $\pm$ 1.86	5
Transformer policy	test_11_20	0.82 $\pm$ 0.02	34.99 $\pm$ 1.37	5
BFS	test_21_40	0.27 $\pm$ 0.06	80.82 $\pm$ 5.00	5
Hand heuristic	test_21_40	0.96 $\pm$ 0.01	27.57 $\pm$ 2.35	5
Feature-MLP policy	test_21_40	0.58 $\pm$ 0.03	55.59 $\pm$ 3.83	5
Transformer policy	test_21_40	0.52 $\pm$ 0.05	59.82 $\pm$ 5.14	5
BFS	test_41_70	0.20 $\pm$ 0.04	85.69 $\pm$ 1.69	5
Hand heuristic	test_41_70	0.73 $\pm$ 0.03	47.34 $\pm$ 2.38	5
Feature-MLP policy	test_41_70	0.46 $\pm$ 0.03	65.25 $\pm$ 3.20	5
Transformer policy	test_41_70	0.41 $\pm$ 0.05	68.81 $\pm$ 4.19	5
BFS	tune_21_35	0.30 $\pm$ 0.04	78.79 $\pm$ 3.25	5
Feature-MLP policy	tune_21_35	0.63 $\pm$ 0.06	51.56 $\pm$ 4.28	5
Transformer policy	tune_21_35	0.55 $\pm$ 0.05	57.31 $\pm$ 3.81	5

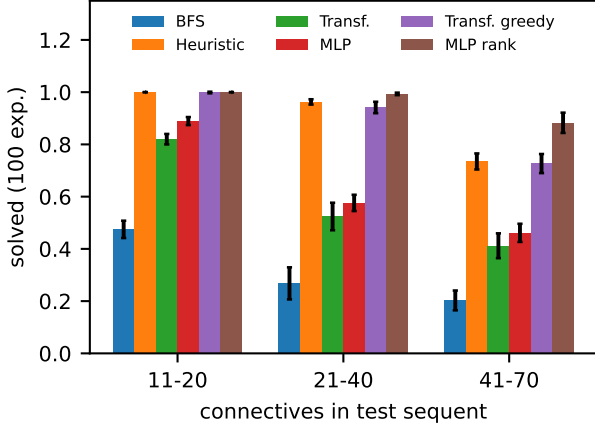


Fig. 1. Solved rate per test-size bin (mean, std over seeds). “Transf.” and “MLP” use $-\log p$ cost; “Transf. greedy” follows the transformer without backtracking; “MLP rank” uses the MLP’s rank as the step cost (post-hoc variants).

the structure of the problem and its expansion count is dominated by the branching factor.

c) H2 (policy > heuristic at 41-70): refuted.: The tuned heuristic solves 0.73 against 0.41 for the transformer (difference -0.3227 , paired $p = 0.0003538$; the delta is transformer minus heuristic, the sign convention of rh compare) and 0.46 for the feature MLP. The heuristic also needs fewer expansions on average (47.3 vs. 68.8). The heuristic is not perfect: it still fails on a large fraction of the 41-70 problems, i.e. these are cases where the budget, not the ordering of non-branching rules, binds.

TABLE III

Diagnostic on heuristic-proof states: top-1 agreement with the heuristic and defer rate (non-branching preferred when both kinds are available). Mean $\pm$ std over seeds; bold/underline marks in this table are not meaningful.

Method	Task	agree_heur $\downarrow$	defer_rate $\downarrow$	n
Transformer policy	val_3_10	0.72 $\pm$ 0.02	0.93 $\pm$ 0.01	5
Feature-MLP policy	val_3_10	0.69 $\pm$ 0.03	0.83 $\pm$ 0.02	5
Transformer policy	test_11_20	0.64 $\pm$ 0.02	0.90 $\pm$ 0.02	5
Feature-MLP policy	test_11_20	0.58 $\pm$ 0.00	0.80 $\pm$ 0.01	5
Transformer policy	test_21_40	0.58 $\pm$ 0.02	0.86 $\pm$ 0.04	5
Feature-MLP policy	test_21_40	0.48 $\pm$ 0.01	0.72 $\pm$ 0.02	5
Transformer policy	test_41_70	0.55 $\pm$ 0.02	0.84 $\pm$ 0.04	5
Feature-MLP policy	test_41_70	0.40 $\pm$ 0.03	0.61 $\pm$ 0.04	5

d) H3 (gap shrinks with size): refuted.: The transformer-minus-heuristic gap in solved rate is -0.18 (11-20), -0.44 (21-40) and -0.32 (41-70). The gap is not monotone: it is smallest at 11-20, largest at 21-40, and narrower at 41-70. Part of the narrowing at 41-70 is a ceiling effect, because the heuristic itself drops to 0.73. We therefore cannot claim that the policy degrades faster than the heuristic with size; we can only say the policy is behind in every bin. (H3 was formulated as a monotone decrease of the policy-minus-heuristic difference, so the pre-registered criterion is not met.)

e) Where the transformer fails.: The transformer is below the MLP in every test bin (paired differences MLP minus transformer: 0.07, 0.05, 0.05; $p = 0.0002829$, 0.02790, 0.02839), although the MLP sees only hand-chosen features. The sequence model has lower validation accuracy and, in the largest bin, also the largest variance between seeds (std 0.05). Per-regime numbers in Table II show that the failure is a size effect for all learned systems: their solved rate falls from the training range to 41-70 much faster than the heuristic’s does at 11-20 and 21-40.

f) Diagnostic: do the policies defer branching rules?: The heuristic’s central principle is to apply non-branching tactics before branching ones. On the states visited by the greedy heuristic proof of each test problem we measured how often the policy’s top-1 tactic is non-branching among states that offer both kinds (defer rate) and how often it equals the heuristic’s top-1 tactic (Table III). The transformer’s defer rate stays high in all bins, while the MLP’s falls strongly with size; the two policies’ top-1 agreement with the heuristic is only moderate even in the training range (which is expected, because many tactics tie in optimal size). These measurements describe the policies’ behaviour; they do not by themselves show that deferring branching explains the search outcomes.

VI. Ablations and Analysis

a) Search mode and cost function (Table IV).: Greedy decoding of the transformer is much better than best-first search on its cumulative $-\log p$ at every size (solved at 41-70: 0.73 vs. 0.41; paired difference 0.31,

TABLE IV

Search-mode and cost ablations (post-hoc variants of the main systems; rank cost: step cost is the tactic’s rank under the policy; greedy: top-1 only). Same trained models as the main table.

Method	Task	solved $\uparrow$	mean_exp $\downarrow$	n
Transformer greedy	test_11_20	1.00 $\pm$ 0.00	11.89 $\pm$ 0.31	5
Transformer rank cost	test_11_20	1.00 $\pm$ 0.00	11.39 $\pm$ 0.29	5
MLP rank cost	test_11_20	1.00 $\pm$ 0.00	9.07 $\pm$ 0.30	5
Transformer greedy	test_21_40	0.94 $\pm$ 0.02	27.96 $\pm$ 2.93	5
Transformer rank cost	test_21_40	0.94 $\pm$ 0.02	26.60 $\pm$ 3.27	5
MLP rank cost	test_21_40	0.99 $\pm$ 0.00	19.51 $\pm$ 2.43	5
Transformer greedy	test_41_70	0.73 $\pm$ 0.04	45.84 $\pm$ 3.17	5
Transformer rank cost	test_41_70	0.73 $\pm$ 0.03	44.58 $\pm$ 3.36	5
MLP rank cost	test_41_70	0.88 $\pm$ 0.04	33.77 $\pm$ 3.24	5

$p = 0.0002448$; at 21–40 0.42, $p = 9.568 \times 10^{-5}$; at 11–20 0.18, $p = 4.300 \times 10^{-5}$). H4 is therefore refuted: backtracking search with $-\log p$ cost is worse, not better, than following the policy. Replacing only the cost by the policy’s rank (same trained model) gives 0.73 at 41–70, statistically indistinguishable from the heuristic there (paired $p = 0.7943$; difference -0.003). Because the model and training are identical in these runs and only the step cost changes, the loss between the main transformer rows and these rows is attributable to the cost function. We did not test why $-\log p$ is a poor cost; one plausible reason (not isolated here) is that cumulative negative log-probability grows with proof depth, so a long proof of $\approx$ tens of steps with small per-step costs competes with many shallow alternatives, which wastes expansions under a tight budget.

b) Post-hoc finding: MLP with rank cost.: The feature MLP with rank cost solves 0.88 at 41–70, 0.99 at 21–40 and 1.00 at 11–20, against the heuristic’s 0.73, 0.96 and 1.00. The paired differences (MLP rank minus heuristic) are 0.15 ($p = 0.001304$) at 41–70 and 0.031 ($p = 0.003552$) at 21–40; at 11–20 both are at the ceiling. Relative to the MLP with $-\log p$ cost, the rank cost adds 0.42 at 41–70 ($p = 4.663 \times 10^{-5}$). This is the only configuration that beats the heuristic, it was not among the registered hypotheses, and it was selected after seeing the test bins from a small family of four policy/cost combinations; the result should be read as a hypothesis for follow-up rather than a confirmed advantage. It indicates that a learned ranking, used with a cost that does not penalise depth, can improve on the heuristic’s ordering when the budget binds, even though the MLP’s defer rate in Table III is lower than the heuristic’s.

c) Learned instead of sinusoidal positions.: Replacing the sinusoidal encoding by learned absolute positions (positions beyond the training length are untrained, indices are clipped at the maximum) leaves the transformer’s solved rate essentially unchanged: 0.83, 0.52, 0.41 for the three bins versus the main rows in Table II (paired differences 0.007, -0.008 , -0.003 ; $p = 0.5281$, 0.4375, 0.8403). Positional extrapolation is thus not the bottleneck we can detect; this does not exclude other forms of length-

TABLE V

Sweeps (solved rate, mean $\pm$ std over 5 seeds). Top: training sequents for the transformer (the 20000 column is the main-table row). Bottom: expansion budget at 41–70 connectives (the budget-100 column is the main-table row).

Training sequents	1000	4000	10000	20000
bin 11–20	0.94 $\pm$ 0.02	0.85 $\pm$ 0.02	0.82 $\pm$ 0.01	0.82 $\pm$ 0.02
bin 41–70	0.52 $\pm$ 0.06	0.42 $\pm$ 0.03	0.41 $\pm$ 0.05	0.41 $\pm$ 0.05
Budget (bin 41–70)	25	50	100	200
Transformer	0.27 $\pm$ 0.05	0.35 $\pm$ 0.05	0.41 $\pm$ 0.05	0.46 $\pm$ 0.04
Heuristic	0.46 $\pm$ 0.04	0.57 $\pm$ 0.03	0.73 $\pm$ 0.03	0.88 $\pm$ 0.02
BFS	0.11 $\pm$ 0.02	0.16 $\pm$ 0.01	0.20 $\pm$ 0.04	0.25 $\pm$ 0.04

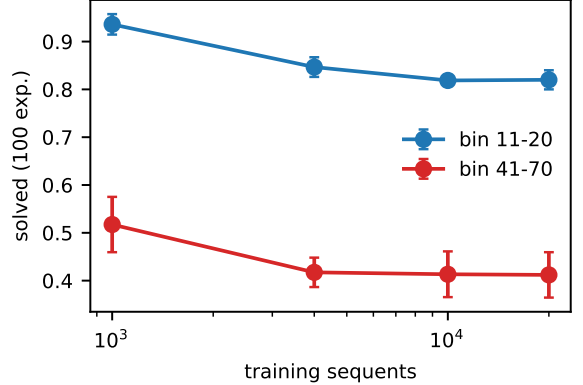


Fig. 2. Transformer solved rate vs. number of training sequents (mean $\pm$ std over seeds).

related failure in the encoder.

d) Training-set size and budget (Table V, Figs. 2–3): Training-set size was varied with a fixed number of optimizer steps, so smaller sets are trained for more epochs. Contrary to the usual expectation, less data extrapolated better: the transformer’s solved rate at 11–20 falls from 0.94 (1000 sequents) to 0.82 (20000), and at 41–70 from 0.52 to 0.41, with the largest step between 1000 and 4000 sequents (Table V). The 1000-sequent policy still stays below the heuristic in both bins. This sweep is exploratory (we did not register it, ran it only with the same steps and learning rate, and did not log in-distribution accuracy for the smaller sets), so we do not know whether the effect is due to overfitting to the larger sets’ structure, to the number of epochs, to the particular states in the smaller sets, or to noise in the optimiser; it does show that the in-distribution data volume is not the limiting factor for size extrapolation here. For the budget sweep at 41–70, all three systems improve with budget; the heuristic’s solved rate is 0.46, 0.57, 0.73, 0.88 for budgets 25, 50, 100, 200, the transformer’s 0.27, 0.35, 0.41, 0.46, and BFS’s 0.11, 0.16, 0.20, 0.25. The ordering heuristic $>$ transformer $>$ BFS in mean solved rate holds at every budget tested; the transformer’s gap to the heuristic does not close with larger budgets in this range.

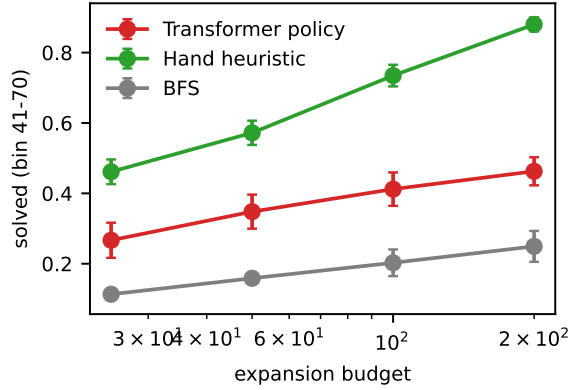


Fig. 3. Solved rate vs. expansion budget in the 41–70 bin (mean $\pm$ std over seeds).

VII. Limitations

- Scale and statistics. Tiny models, 2000 steps, one training set size, 5 seeds and 150 problems per task and seed. Paired t -tests with five seeds are underpowered for small effects and are uncorrected for multiple comparisons; “not significant” means “not detected”.
- Synthetic, easy calculus. The rules are invertible, so a wrong tactic never makes a goal unprovable; the policy only affects proof size. This is a much weaker notion of “tactic prediction” than in interactive provers with dead ends, and the conclusions may not transfer to non-invertible systems (intuitionistic logic, first-order logic with instantiation) or to real libraries. The task contains no arithmetic.
- Problem distribution. Four hand-designed generators, rejection to a size range, 5 variables. Sizes of the test sequents are strictly larger than training, but the generators (e.g. substitution instances and rewrites) also shift structure with size, so “size” and “structure” are confounded. Optimal proof size was computed only for training sequents; we did not measure how far from optimal the test proofs of each system are.
- Baseline strength. The heuristic was tuned over three variants on a bin from the target size regime; it is a hand-designed rule that exploits the known structure of G3cp, which is the strongest hand baseline we could write in the time available but is not provably optimal (the MLP with rank cost beats it). The policies were not tuned at all; a tuned policy (longer training, more capacity, other losses, value or proof-size heads, search with learned value) could behave differently.
- Post-hoc analysis. The cost-function and MLP-rank results were run after the main runs revealed the weakness of $-\log p$ search; they are exploratory. We did not isolate why $-\log p$ cost hurts.
- Reimplementation. BFS, the heuristic search and both policies are our reimplementations in a shared

engine; absolute numbers cannot be compared with published provers. Axiom closure is free and the first open goal is always expanded; other goal-selection and expansion-counting conventions would change absolute numbers.

VIII. Conclusion

In a small invertible sequent calculus, a transformer tactic policy trained to imitate optimal tactics on formulas with 3–10 connectives guides best-first search better than breadth-first search on larger formulas (H1 supported) but not better than a tuned hand heuristic (H2 refuted), and its deficit does not shrink in a monotone fashion with size (H3 refuted). Searching on cumulative $-\log p$ is what makes the transformer lose most: the same model decoded greedily or ranked reaches the heuristic’s level at 41–70 connectives (H4 refuted: search with this cost is worse than greedy). Training on fewer sequents (1000 instead of 20000) improved extrapolation, an unexplained exploratory result. A feature MLP with a rank cost exceeded the heuristic in a post-hoc comparison, which we offer as a hypothesis for a larger study. Natural next steps are a pre-registered test of rank versus likelihood costs with more seeds, policies that predict remaining proof size, non-invertible calculi where wrong tactics have a cost, and a measurement of proof-size optimality beyond the solved-within-budget metric.

References

- [1] S. Polu and I. Sutskever, “Generative Language Modeling for Automated Theorem Proving,” arXiv preprint arXiv:2009.03393, 2020. [Online]. Available: <https://arxiv.org/abs/2009.03393>
- [2] G. Lample, M.-A. Lachaux, T. Lavril, X. Martinet, A. Hayat, G. Ebner, A. Rodriguez, and T. Lacroix, “HyperTree Proof Search for Neural Theorem Proving,” arXiv preprint arXiv:2205.11491, 2022. [Online]. Available: <https://arxiv.org/abs/2205.11491>
- [3] R. Xin, C. Xi, J. Yang, F. Chen, H. Wu, X. Xiao, Y. Sun, S. Zheng, and K. Shen, “BFS-Prover: Scalable Best-First Tree Search for LLM-based Automatic Theorem Proving,” arXiv preprint arXiv:2502.03438, 2025. [Online]. Available: <https://arxiv.org/abs/2502.03438>
- [4] K. Yang, A. M. Swope, A. Gu, R. Chalamala, P. Song, S. Yu, S. Godil, R. Prenger, and A. Anandkumar, “LeanDojo: Theorem Proving with Retrieval-Augmented Language Models,” arXiv preprint arXiv:2306.15626, 2023. [Online]. Available: <https://arxiv.org/abs/2306.15626>
- [5] A. A. Alemi, F. Chollet, N. Eén, G. Irving, C. Szegedy, and J. Urban, “DeepMath - Deep Sequence Models for Premise Selection,” arXiv preprint arXiv:1606.04442, 2016. [Online]. Available: <https://arxiv.org/abs/1606.04442>
- [6] S. Loos, G. Irving, C. Szegedy, and C. Kaliszzyk, “Deep Network Guided Proof Search,” arXiv preprint arXiv:1701.06972, 2017. [Online]. Available: <https://arxiv.org/abs/1701.06972>
- [7] C. Kaliszzyk, J. Urban, H. Michalewski, and M. Olśák, “Reinforcement Learning of Theorem Proving,” arXiv preprint arXiv:1805.07563, 2018. [Online]. Available: <https://arxiv.org/abs/1805.07563>
- [8] E. Aygün, L. Orseau, A. Anand, X. Glorot, V. Firoiu, L. M. Zhang, D. Precup, and S. Mourad, “Proving Theorems using Incremental Learning and Hindsight Experience Replay,” arXiv preprint arXiv:2112.10664, 2021. [Online]. Available: <https://arxiv.org/abs/2112.10664>

- [9] D. Saxton, E. Grefenstette, F. Hill, and P. Kohli, “Analysing Mathematical Reasoning Abilities of Neural Models,” arXiv preprint arXiv:1904.01557, 2019. [Online]. Available: <https://arxiv.org/abs/1904.01557>
- [10] P. Clark, O. Tafjord, and K. Richardson, “Transformers as Soft Reasoners over Language,” arXiv preprint arXiv:2002.05867, 2020. [Online]. Available: <https://arxiv.org/abs/2002.05867>
- [11] C. Anil, Y. Wu, A. Andreassen, A. Lewkowycz, V. Misra, V. Ramasesh, A. Slone, G. Gur-Ari, E. Dyer, and B. Neyshabur, “Exploring Length Generalization in Large Language Models,” arXiv preprint arXiv:2207.04901, 2022. [Online]. Available: <https://arxiv.org/abs/2207.04901>
- [12] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention Is All You Need,” arXiv preprint arXiv:1706.03762, 2017. [Online]. Available: <https://arxiv.org/abs/1706.03762>